# Real-Time Parallel Log / Event Triage Engine

*A Regex-to-DFA Compiler for Streaming Log Classification: Review 1 Report*

Adithya Santhosh P
Reg. No. 24BCI0099

Anvesha Tripathi
Reg. No. 24BCT0055

*Team 1, Compiler Design Course Project, VIT · Review 1 · October 2026*

***Abstract*—Servers emit thousands of log lines per second, and the few that matter—repeated failed logins, injection attempts, crashes, leaked personal data—must be found as the lines arrive. This report presents the first milestone of a real-time log triage engine: a purpose-built regular-expression compiler and a baseline classification pipeline. Twenty-six detection rules are parsed by a recursive-descent parser, converted to a Thompson NFA, and compiled by subset construction into combined DFAs that match every rule in one linear pass over a line. The compiled DFA agrees with the C library's regex engine on all 10,179 corpus lines and 100,000 generated strings (0 mismatches), and it processes 528,283 lines/s against 64,778 lines/s for the POSIX engine (8.2× faster, one thread, end to end). Splitting the rules into groups reduces the DFA from 39,764 states to 7,593. The report also reproduces and measures the course's base paper, whose process-per-pattern design is slower than a sequential loop, while splitting the input across cores scales. Checksum validation, confidence scoring, multi-core execution and attack correlation are the next milestones.**

***Index Terms*—**regular expressions, finite automata, subset construction, log classification, deep packet inspection, multicore processing.

## 1. INTRODUCTION

### *1.1 Motivation*

Modern services write logs faster than people can read them. Hidden in the stream are lines that need action: repeated authentication failures, SQL-injection or path-traversal requests, application crashes, slow queries, and personal data that must not be stored in clear. *Triage*—assigning every line a category and a severity, and routing urgent lines to an alert stream—has to keep up with the input rate. Hand-written regular-expression rules are the traditional way to encode this knowledge [1], which makes the matching engine the performance-critical part.

The base paper given for this course classifies short text inputs by running one regular expression per forked process on separate cores. We treat its design and claims as hypotheses and test them. This report follows a different route to speed: compile all rules into one deterministic automaton, then (in later milestones) parallelise across input lines rather than across patterns.

### *1.2 Problem Statement and Objectives*

*Problem.* Given a stream of log lines and a set of at most 64 rules, each a regular expression with a category, a severity and a base score, assign every line the categories whose rules match it, a primary category and a severity, at a rate that keeps up with the input. The matching semantics must equal POSIX extended regular expressions on the supported subset, so that the C library can serve as an independent reference, and a rule outside the subset must be reported, never silently approximated.

*Objectives.* (O1) Build a regex-to-DFA compiler from first principles: lexer, parser, Thompson NFA and subset construction. (O2) Verify it against the C library by differential testing. (O3) Provide a single-threaded pipeline that classifies real logs with two interchangeable engines. (O4) Measure the base paper's design and fairer alternatives, including our DFA, under a documented protocol. Later milestones add validation, confidence scoring, multi-core execution and attack correlation.

### *1.3 Contributions of This Milestone*

1) a **regex front end**: lexer and recursive-descent parser for the regular POSIX-ERE subset that rejects anything outside it;
2) a **regex-to-DFA compiler**: Thompson NFA, subset construction with accept masks and automatic rule grouping, and a table-driven matcher;
3) a **baseline pipeline**: rule loader, normaliser, batching reader, two interchangeable engines (our DFA and the C library), first classifier and JSON-lines output;
4) a faithful **reproduction and benchmark** of the base paper, with fairer baselines and our DFA as one of the variants.

### *1.4 Organization*

Fig. 1 gives an overview of the architecture: the rules are compiled once at start-up by the front end and the DFA compiler, the pipeline then streams lines through matching and classification, and the evaluation harness measures everything. Section 2 surveys related work, Section 3 describes each module with pseudocode, Section 4 gives the implementation and per-module results, Section 5 compares existing and proposed methods, and Section 6 concludes.

### *1.5 Scope and Limitations*

This report covers the project at commit `8093e0a` only. Classification at this stage is rule based: a matched rule contributes its category, with no validation of the matched text, no confidence score and no masking of personal data, and the effect is visible in Section 4.4, where digit identifiers are labelled as personal data. The pipeline is single-threaded, so the multi-core results of Section 5 concern the five-pattern benchmark of the base paper, not the log pipeline itself. Accuracy figures exist for the five-pattern task, for which a labelled set was built; for the log rules we report agreement with annotated lines and with the C library, not precision and recall against human labels. The sample logs are small (2,000 lines each), so the throughput experiment repeats them, and all absolute numbers depend on the machine described in Section 4.

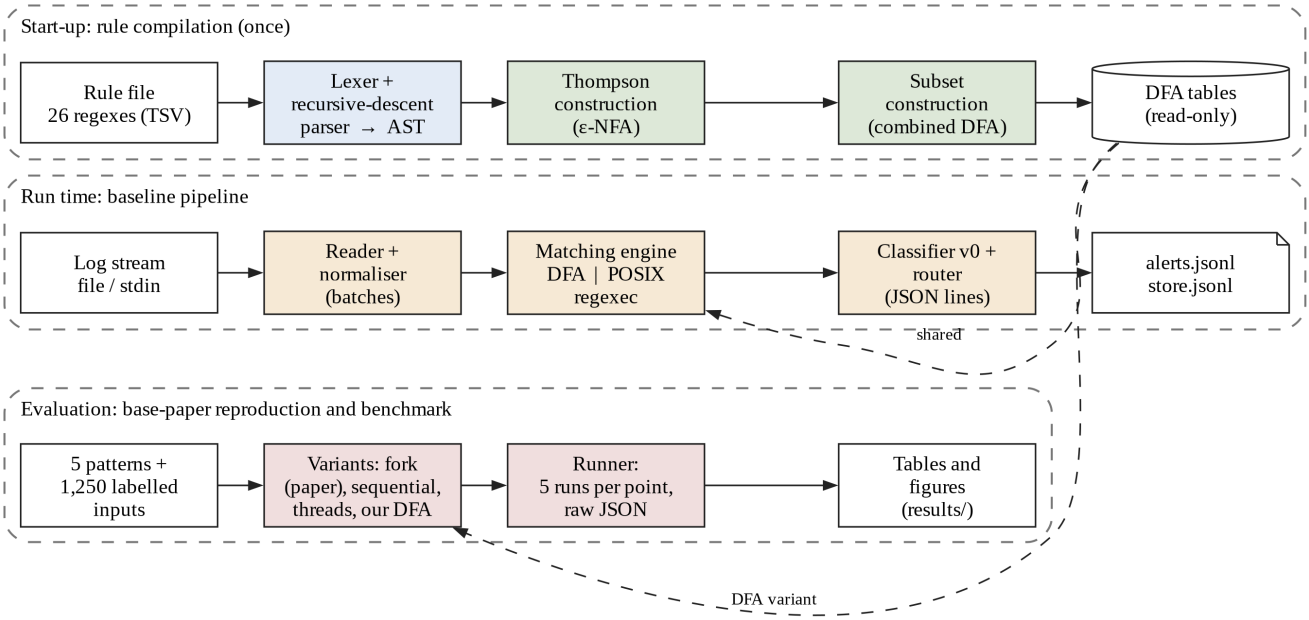



Fig. 1. System architecture: start-up compilation, baseline pipeline and evaluation harness.

## 2. LITERATURE SURVEY

### *2.1 Automata Foundations and Regex Compilation*

Rabin and Scott introduced nondeterministic finite automata and the decision problems they raise [2], which gives the subset construction its theoretical basis. McNaughton and Yamada gave a procedure that turns a regular expression into an automaton [3]. Thompson described the search algorithm that compiles a regular expression into a nondeterministic matcher [4]; the construction named after him is the one used here. For a fixed set of literal patterns, Aho and Corasick match all patterns in a single pass over the text [5], the same one-pass idea that a combined DFA extends from literals to regular expressions.

### *2.2 Multi-Pattern Matching and DFA State Explosion*

Regular-expression matching for deep packet inspection (DPI) has a large literature. The survey by Xu *et al.* covers applications, algorithms and hardware platforms and identifies the growth in DFA size as the central difficulty [6]. Responses fall into three groups. *Compression:* $D^2FA$ reduces memory with default transitions [7]; differential encoding of DFAs keeps one transition per byte with smaller tables [8]; A-DFA bounds the number of default-transition hops [9]. *Hybrids and decomposition:* hybrid automata combine a DFA with an NFA for complex intrusion-detection patterns [10], and the extensions to counting and back-references are treated in [11]; Liu and Norige decompose expressions to avoid the explosion [12]. *Prefiltering and engineering:* Bloom-filter-assisted matching for intrusion detection [13], and Hyperscan, which splits each expression into literal and automaton parts and uses SIMD matching, reporting an 8.7× speed-up for Snort on a real trace [14]. Backtracking engines have a different weakness: Davis *et al.* found super-linear regular expressions in more than 10,000 JavaScript and Python modules [15], which is the argument for an automaton engine whose cost is linear in the input. Our grouping strategy (split the rule set in halves until each DFA fits a state cap) is deliberately simpler than these techniques.

### *2.3 Parallel Pattern Matching and Concurrency*

Luchaup *et al.* parallelise the matching of *one* input by splitting it into chunks and speculating on the start state of each [16]; Ko *et al.* study a speculative parallel DFA membership test for multicore, SIMD and cloud settings [17], and Mytkowicz *et al.* make finite-state machines data parallel by computing transitions for all possible start states, reporting up to 21× on eight cores [18]. Yang and Prasanna show that large automata behave poorly in the cache of a multicore processor [19]. Log lines are already independent units, so line-level parallelism needs no speculation; this is the option adopted for the next milestone. Two performance-theory points frame the experiments: Gustafson's scaled-speedup argument [20], and Lamport's work on specifying the interfaces of concurrent program modules [21], relevant to the planned hand-off between pipeline stages. Baumann *et al.* argue that `fork()` has become a poor primitive for modern systems [22], which is relevant to a base-paper design that forks per pattern per input.

### *2.4 Log Analysis, Datasets and Sensitive Data*

He *et al.* survey automated log analysis for reliability engineering [1]. Log parsers extract templates: Drain uses a fixed-depth tree for online parsing [23], and POP parallelises parsing on a cluster [24]. Landauer *et al.* survey clustering for security log analysis and group approaches into overview and filtering, parsing and signature extraction, outlier detection, and sequence anomaly detection [25]; a rule-based triage engine belongs to the filtering and signature group. Public data: Loghub collects system logs from many software systems [26], and the AIT data sets add labelled attack steps [27] produced by a simulation testbed [28]. Event processing platforms are surveyed in [29]. For personal data, the Text Anonymization Benchmark provides an annotated corpus of 1,268 court cases [30] and Lison *et al.* review anonymisation models [31]; check-digit schemes such as the one studied by Gumm [32] allow identifier-like numbers to be validated by arithmetic.

### 2.5 Benchmarking Methodology

Hoefler and Belli reviewed 120 papers from three high-performance-computing conferences, found that reporting of parallel performance was often inadequate, and propose rules for it [33]; Roodt *et al*. benchmark regular-expression engines built on Thompson and Glushkov constructions [34]. We follow their spirit: every data point is the median of five runs with minimum and maximum reported, the machine is described, and nothing is tuned to reproduce a published number.

### 2.6 Summary and Research Gap

TABLE I
SURVEYED WORK AND ITS RELATION TO THIS PROJECT

| Work | Technique | Relation to this work |
|---|---|---|
| [6] | DPI regex survey | State explosion is the key issue |
| [7]–[9] | DFA compression | Smaller tables; not needed yet |
| [10], [12] | Hybrid / decomposed automata | More complex than our grouping |
| [14] | Literal prefilter + SIMD | Industrial engine, comparison target |
| [15] | ReDoS study | Motivates linear-time DFA |
| [16]–[18] | Parallel DFA on one input | We split lines, not inputs |
| [1], [25] | Log analysis surveys | Rules = filtering / signatures |
| [26], [27] | Log data sets | Our test data |
| [22], [33] | fork cost; benchmarking | Base-paper analysis, method |

In the surveyed work, regex automata research targets network payloads, while log research targets parsing, clustering and anomaly detection. A compiled multi-rule DFA applied to per-line triage of heterogeneous logs, evaluated against a process-parallel baseline, is the gap this project addresses.

## 3. PROPOSED SYSTEM

The engine is one C11 program with two phases (Fig. 1). At start-up, the rule file is compiled into DFA tables by the regex front end and the DFA compiler. At run time, the baseline pipeline reads log lines, normalises them, matches them with either engine, assigns a category and severity, and writes JSON-lines records. The evaluation harness reproduces the base paper and measures it together with our DFA. Table II gives the module split.

TABLE II
MODULE SPLIT AT THIS MILESTONE

| Module | Source files | New | Lines |
|---|---|---|---|
| Regex front end | regex_lex, regex_parse, ast | yes | 1,024 |
| NFA/DFA compiler | nfa, dfa, dot, engine_dfa | yes | 1,352 |
| Baseline pipeline | rules, reader, normalize, timestamp, batch, engine, engine_posix, aggregate, route, json, pipeline, cli, main, types | no | 2,446 |
| Reproduction and benchmark | tools/basepaper/ (fork, variants, runner, evaluation) | yes | 2,099 |

The regex front end and the DFA compiler are the new contributions of the compiler and carry full pseudocode; the pipeline and the evaluation harness are described with shorter pseudocode. The engine core totals 4,822 lines of C in 40 files.

### 3.1 Regex Front End

The front end turns the text of a rule's regular expression into an abstract syntax tree (AST). The grammar is the POSIX-ERE subset—literals, `.`, bracket expressions, `* + ? {n,m}`, alternation, grouping and the anchors `^ $`—so the same text also runs on the C library, which makes differential testing possible. A lexer produces tokens and the parser has one function per non-terminal (Algorithm 1). Character classes are 256-bit sets, and case-insensitive rules fold the set once at parse time. Every construct outside the regular subset is rejected with the byte offset of the offending character; nothing is approximated, because an approximation would silently change what a rule matches.

**Algorithm 1. ParseRegex (recursive descent)**

```
function Parse(pattern, icase):
    ast ← ParseAlt(); if input remains then Error(pos)
    return ast
function ParseAlt():
    kids ← [ParseConcat()]
    while next = '|' do append ParseConcat() to kids
    return Alt(kids) if |kids| > 1 else kids[0]
function ParseConcat():
    while next is not '|', ')' or END do append ParseRepeat()
    return Concat(kids) (Empty if none)
function ParseRepeat():
    node ← ParseAtom()
    while next is a quantifier (*, +, ?, {n,m}) do
        if a '?' or '+' follows the quantifier then Error(lazy)
        if {n,m}: need n ≤ m ≤ 1000, else Error
        node ← Repeat(node, n, m)
    return node
function ParseAtom():
    '(' : if next = '?' then Error(lookaround); depth ≤ 100;
        node ← ParseAlt(); expect ')' (else Error(unbalanced))
    '[' : read class into a 256-bit set; '.' : all bytes except \n
    '\' : only escaped metacharacters; \d \b \1 ... → Error
    '^', '$' : only at pattern start / end, else Error
    other : literal byte (set folded if icase)
```

### 3.2 NFA and Combined-DFA Construction

The DFA compiler compiles the ASTs of all rules into one automaton. Thompson's construction [4] gives every AST node a small NFA fragment with one entry and one exit (Algorithm 2); a bounded repetition `x{n,m}` is expanded into copies. The fragments of all rules are joined under one start state, and each accept state is tagged with its rule number. Subset construction [2] then builds a DFA whose states are sets of NFA states (Algorithm 3). Three design points matter. First, each DFA state stores an *accept mask*, a 64-bit word whose bit *r* says that rule *r* has matched, so one pass over a line reports every matching rule. Second, to find a match anywhere in the line, the set *U* of start states of all non-anchored rules is added to every transition; anchors are handled by assertion edges that only fire at offset 0 (`^`) or at the end of the line (`$`). Third, bytes that no character set of the NFA distinguishes are merged into byte classes, so the transition function is computed once per class. When a DFA would exceed a state cap, the rule set is halved and each half is built separately, recursively; the group results are OR-ed at match time. Matching is the loop of Algorithm 4.

**Algorithm 2. Thompson construction (fragment = entry, exit)**

```
function Build(node) → (s, e):
    Charset: new s, e; s —set→ e
    Concat(k_1…k_n): link exit(k_i) —ε→ entry(k_{i+1})
    Alt: split chain; s —ε→ entry(k_i); exit(k_i) —ε→ e
    Star: s —ε→ entry(k), e; exit(k) —ε→ entry(k), e
    Plus: as Star, but s = entry(k)
    Quest: s —ε→ entry(k), e; exit(k) —ε→ e
    Repeat{n,m}: n copies in sequence, then (m−n) nested optional copies
    ^ , $ : assertion edge, usable at offset 0 / at line end only
for each rule r: (s_r, e_r) ← Build(ast_r); tag e_r accept(r)
start ← new state with ε-edges to every s_r
```

**Algorithm 3. Subset construction with accept masks**

```
function BuildDfa(rules, cap):
    U ← ε-closure of the start states of rules not '^'-anchored
    S_0 ← ε-closure of the start states of all rules, '^' allowed
    table ← {S_0 : 0}; work ← [S_0]
    while work not empty do
        S ← pop(work)
        for each byte class c do
            T ← ε-closure(move(S, c)) union U
            if T not in table then
                if |table| = cap then return TOO_BIG
                add T to table; push T on work
            next[S][bytes of c] ← id(T)
        accept[S] ← rules whose accept state is in S
        accept_end[S] ← rules reachable from S by ε / '$' edges
    return tables
function BuildAll(rules):
    if BuildDfa(rules) = TOO_BIG then
        if |rules| = 1 then Error
        BuildAll(first half); BuildAll(second half)
```

**Algorithm 4. DFA match loop**

```
function Match(line) → mask:
    mask ← 0
    for each group DFA D do
        st ← 0; m ← accept[0]
        for each byte b of line do
            st ← next[st][b]; m ← m | accept[st]
        mask ← mask | m | accept_end[st]
    return mask
```

### *3.3 Baseline Pipeline*

The baseline is a single-threaded pipeline that makes the compiler usable and gives an independent reference. The *rule loader* reads a tab-separated file (id, category, severity, base score, flags, validator, extract list, context keywords, regex) and rejects malformed rows with file and line number. The *reader* splits a file or stdin into lines, handles CRLF, a final line without newline and very long lines, parses syslog or ISO-8601 timestamps and fills batches. The *normaliser* makes the match copy of each line: percent escapes are decoded (so `%27` is seen as an apostrophe), '+' becomes a space and control characters are removed, because attack strings in web logs are usually URL-encoded. Two engines sit behind one interface: the DFA engine described above and a POSIX engine that calls the C library's `regexec` once per rule. The first classifier assigns each matched line the categories of its rules, the primary category by highest severity, and writes JSON-lines records to a store file and an alert file (severity HIGH or above).

**Algorithm 5. Baseline pipeline**

```
function Run(rulefile, input, engine):
    rules ← LoadRules(rulefile); E ← Compile(rules, engine)
    while batch ← ReadBatch(input, B) do
        for each line i in batch do
            norm ← Normalize(raw_i) // %xx, '+', control chars
            mask ← E.Match(norm)
            labels ← categories of rules in mask
            primary ← label of highest severity (tie order fixed)
            write JSON record to store; if severity ≥ HIGH: also to alerts
```

### *3.4 Base-Paper Reproduction and Benchmark*

The evaluation harness rebuilds the base paper's program as described and measures it. For each input the program creates a pipe, forks one child per pattern, pins each child to a core, and the child compiles its regular expression, matches it and reports the result through the pipe (Algorithm 6); the parent collects the five records and reaps the children. The paper's input-normalisation pseudocode is reproduced as an option, and its inefficiencies are kept on purpose. For comparison, one benchmark binary offers six variants on the same five patterns: the forking design, a sequential loop that recompiles per input, a sequential loop with precompiled patterns, five threads (one pattern each), data-parallel threads (inputs split across threads), and our combined DFA with split inputs. A runner executes every experiment five times and stores the raw JSON.

**Algorithm 6. Base-paper classification (fork per pattern)**

```
function ClassifyFork(text):
    create pipe; for i in 0…4 do
        pid_i ← fork()
        if child then
            pin to CPU (i mod ncpu); regcomp(P_i)
            ok ← regexec(text); regfree
            write (i, ok, t_compile, t_exec) to pipe; exit
    for k in 1…5 do
        read one record; waitpid(pid_k)
    return bit mask of matched patterns
```

## 4. RESULTS AND IMPLEMENTATION

All results come from commit `8093e0a`, built with gcc 15.2 and `-O2 -Wall -Wextra -Wpedantic -pthread` without warnings, on a 13th-generation Intel Core i7-13650HX (14 cores) under Linux. The code base has 4,822 lines of engine source, 3,518 lines of tests and 2,099 lines of base-paper tooling. Every measurement is stored under `results/`.

### *4.1 Dataset Description*

TABLE III
DATA SETS

| Data set | Source | Lines | Use |
|---|---|---|---|
| OpenSSH_2k | Loghub sample | 2,000 | AUTH rules |
| Apache_2k | Loghub sample | 2,000 | web-server log |
| Linux_2k | Loghub sample | 2,000 | syslog / PAM |
| BGL_2k | Loghub sample | 2,000 | ERROR rules |
| HDFS_2k | Loghub sample | 2,000 | large-volume log |
| synthetic_small | hand-written | 179 | 83 annotated lines: positives, near-misses |
| labelled set | generated, fixed seed | 1,250 | five base-paper categories |

The first five are the 2,000-line samples published with Loghub [26]. They carry no labels, so for them we report what the engine finds. The hand-written corpus annotates each line with

the exact set of rules that must match it, including look-alikes such as a 12-digit order number or the word “errors”. The labelled set of 1,250 inputs is generated from templates with a fixed seed, balanced over e-mail, phone, question, greeting and name plus none, multi-category and near-miss lines; its ground truth describes what a line *is*, not what the regexes match.

### *4.2 Regex Front End*

TABLE IV
REJECTED SYNTAX (MESSAGE AND BYTE OFFSET)

| Pattern | Compiler message | Offset |
|---|---|---|
| `a\d+` | \d shorthand not supported (write [0-9]) | 1 |
| `a\bb` | \b word boundary not supported | 1 |
| `(a)\1` | backreference \1 (not a regular language) | 3 |
| `(?=a)b` | "(?" groups: lookaround not supported | 0 |
| `a*?b` | lazy / possessive quantifier | 2 |
| `a{,3}` | {,n} not supported; write {0,n} | 1 |
| `a^b` | '^' only allowed at pattern start | 1 |
| `(ab` | unbalanced '(' | 0 |
| `a{2,1}` | invalid interval, m < n | 1 |

All 26 default rules parse. The parser test has 97 valid and invalid cases (264 checks). Table IV shows that every unsupported construct is refused with a message that names a regular alternative where one exists. *Inference:* the front end enforces the regular subset strictly; a rule author is told how to rewrite the rule instead of getting silently different behaviour.

### *4.3 NFA and DFA Construction*

The 26 rules give a combined NFA of 1,762 states. Table V and Fig. 2 show what the state cap does. Without a cap the combined DFA has 39,764 states and takes 1125 ms to build; with the default cap of 20,000 it is split into 2 groups with 7,593 states in total (600 ms), a 5.2× reduction in table states. Tighter caps give more groups: at 500 states the DFA needs 5 groups of 1,680 states in total, built in 37 ms.

TABLE V
DFA SIZE VERSUS STATE CAP (26 RULES)

| Cap | Groups | DFA states | Build (ms) |
|---|---|---|---|
| 500 | 5 | 1,680 | 37 |
| 1,000 | 5 | 1,680 | 70 |
| 2,000 | 4 | 2,899 | 111 |
| 5,000 | 3 | 5,708 | 251 |
| 20,000 | 2 | 7,593 | 600 |
| 1,000,000 | 1 | 39,764 | 1125 |

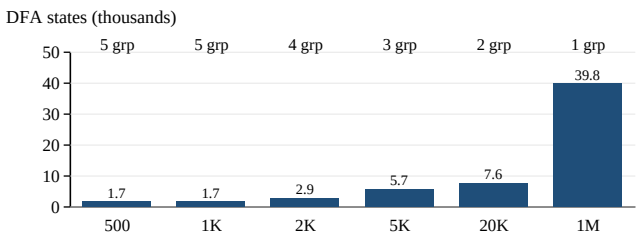



Fig. 2. Total DFA states against the state cap; the label above each bar is the number of groups.

*Correctness.* The differential tester compares the DFA's rule mask with the C library's for every rule on every line of the 6 corpus files (10,179 lines) and on 100,000 generated strings: **0 mismatches**. The DFA test (`169,082` checks) additionally compares the DFA with an NFA simulator, and the NFA test (147,063 checks) compares that simulator with `regexec`. *Inference:* the combined DFA is semantically equal to the reference, and grouping trades a few more table lookups per byte (one pass per group) for much smaller tables.

### *4.4 Baseline Pipeline*

TABLE VI
CLASSIFICATION OUTPUT AT THIS MILESTONE

| File | Lines | Rule hits | Primary label counts |
|---|---|---|---|
| Apache_2k | 2,000 | 0 | none |
| BGL_2k | 2,000 | 405 | ERR 389, PII 9, PERF 7 |
| HDFS_2k | 2,000 | 119 | PII 119 |
| Linux_2k | 2,000 | 30 | PII 29, PERF 1 |
| OpenSSH_2k | 2,000 | 1,128 | AUTH 1125, ERR 2 |
| synthetic_small | 179 | 55 | PII 14, SEC 13, ERR 10, AUTH 8, PERF 8 |

The two engines produce the same labels, and both reproduce the annotated rule set of all 83 annotated lines of the hand-written corpus; the end-to-end test compares the pipeline output with an expected file for both engines. Table VI shows the distribution on real logs: OpenSSH is dominated by authentication failures (1,125 of 2,000 lines) and BGL by errors. *Inference and limitation:* the 119 lines labelled PII in HDFS are block and task identifiers (for example `blk_278357163850888`), long digit strings that match the card and Aadhaar patterns. Pattern matching alone cannot tell an identifier from a card number; checksum validation of the matched digits is the planned remedy, and the same effect explains the PII hits in the Linux sample.

### *4.5 Base-Paper Reproduction*

TABLE VII
LATENCY PER INPUT, MEDIAN (MS)

| Input | fork | seq-rec. | seq-pre. | thr-pat. | DFA |
|---|---|---|---|---|---|
| `john@example.c…` | 193 | 15.4 | 0.364 | 5.23 | 0.045 |
| `9876543210` | 197 | 15.7 | 0.321 | 5.18 | 0.034 |
| `What is your n…` | 201 | 15.1 | 0.673 | 4.97 | 0.05 |
| `hello everyone` | 193 | 15.7 | 0.66 | 5.31 | 0.042 |
| `John Smith` | 205 | 15.8 | 0.308 | 5.12 | 0.034 |

The reproduced program classifies one input in 193–205 µs, while a sequential loop that recompiles the five patterns needs 15–16 µs (Table VII). Inside each child, `regcomp` takes about 20 µs and `regexec` about 3–5 µs, so process management is most of the 193 µs. The paper's input module also rejects 146 of the 1,250 labelled lines (for example those containing '+' or an apostrophe), because its character whitelist is narrower than the inputs. *Inference:* the paper's central hypothesis—that process overhead dominates—is confirmed, and it works against the paper's own speed-up claim.

## 5. PERFORMANCE AND METRICS

### *5.1 Matching Engine: Existing versus Proposed*

We compare the C library's POSIX engine (one `regexec` per rule, the approach of the base paper's matching step) with the proposed combined DFA. Both run the same 26 rules inside the same single-threaded pipeline on 1,017,500 log lines (123 MB, the six corpus files repeated 100 times), output written to memory-backed files; the start-up cost of each engine (rule compilation) is measured separately on an empty input and subtracted. Five runs per engine were interleaved.

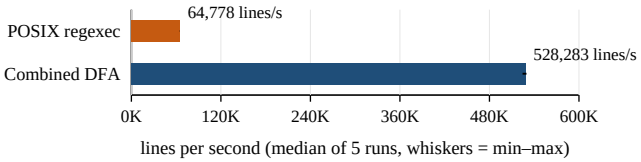



Fig. 3. End-to-end throughput of the two engines on real log rules, one thread.

The DFA engine sustains 528,283 lines/s and the POSIX engine 64,778 lines/s, a factor of **8.2**; the spread between the five runs is at most 1 % of the median. *Inference:* in a loop of 26 library calls every line is scanned once per rule, and the library's cost grows with the rule count; the DFA scans the line once whatever the rule count. The factor is a lower bound on the matching speed-up, because the measurement includes the unchanged reader, classifier and output. The price is the start-up cost: building the DFAs takes 617 ms against 17 ms for compiling the library's regexes, which is repaid after about 44 thousand lines.

### *5.2 Base-Paper Design versus Alternatives*

Fig. 4 compares six designs on the five-pattern task for a batch of 10,000 inputs (median of five runs). The run-to-run spread (maximum minus minimum, relative to the median) is at most 5 % for 4 designs and 45 % and 92 % for the two fastest, whose 10,000-input runs last only a few milliseconds or less and are dominated by timing noise; the scaling runs of Fig. 5 therefore use longer runs, up to 10 million inputs.

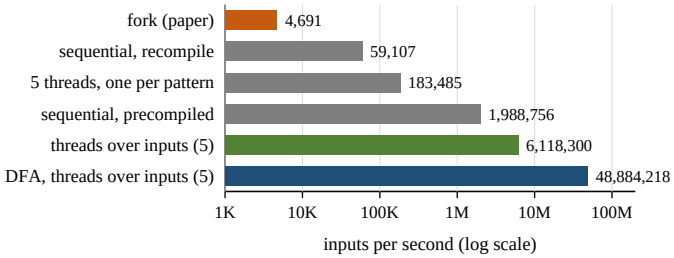



Fig. 4. Throughput of six designs for 10,000 inputs (existing: first row; proposed: last row).

The forking design reaches 4,691 inputs/s, which is 12.6× slower than a sequential loop that recompiles and 424× slower than one with precompiled patterns. Splitting the *inputs* across five threads gives 3.1× the sequential precompiled loop, and our DFA with the same split 24.6×. Fig. 5 shows scaling with cores: the fork design peaks at 1.72× and per-pattern threads at 2.54×, because five patterns cannot give more than five-fold parallelism and the setup cost is paid per input; data-parallel threads reach 4.83× and the DFA variant 5.29× at six cores; the eight-core point is lower because it uses two slower efficiency cores. *Inference:* the base paper's design cannot deliver its claimed near-linear scaling on this hardware; parallelism over input items does.

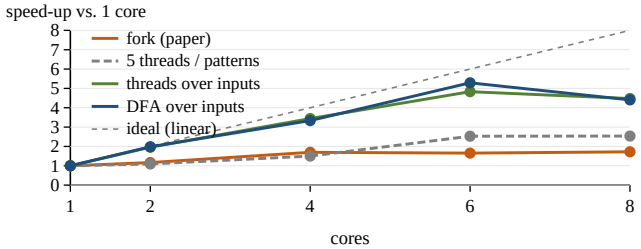



Fig. 5. Speed-up against the same design on one core (median of 5 runs).

### *5.3 Accuracy: Existing versus Proposed*

On the labelled set of 1,250 inputs we compare the base paper's pipeline (its input module followed by `regexec`; *existing*) with matching on the unmodified text with the same five patterns, as done by the DFA variant (*proposed*). The DFA gives the same masks as `regexec` on every line (asserted by a test), so any difference comes from the paper's input module. Table VIII and Fig. 6 give precision (P), recall (R) and F1 per category.

TABLE VIII
ACCURACY ON THE LABELLED SET

| Category | P exist. | R exist. | F1 exist. | P prop. | R prop. | F1 prop. |
|---|---|---|---|---|---|---|
| Email | 0.99 | 0.75 | 0.85 | 0.99 | 1.00 | 0.99 |
| Phone | 0.88 | 0.67 | 0.76 | 0.91 | 0.87 | 0.89 |
| Question | 1.00 | 0.76 | 0.87 | 1.00 | 0.77 | 0.87 |
| Greeting | 1.00 | 0.78 | 0.88 | 1.00 | 0.70 | 0.83 |
| Name | — | 0.00 | 0.00 | 0.77 | 0.67 | 0.71 |
| **Exact match** | | | 0.698 | | | 0.798 |
| **Label accuracy** | | | 0.933 | | | 0.960 |

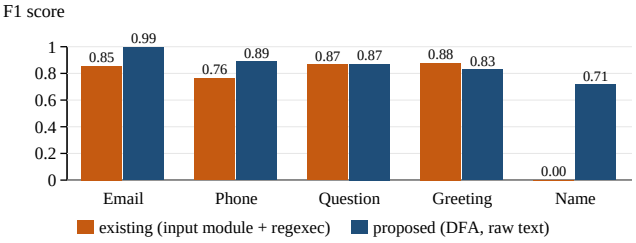



Fig. 6. F1 per category, existing pipeline versus proposed matching.

With the proposed matching, exact-set accuracy is 0.798 against 0.698 for the existing pipeline, and label accuracy 0.960 against 0.933. The existing input module lowercases text, so the name pattern, which needs capital letters, can never match (recall 0.00), and it rejects 146 inputs. Without it, e-mail is nearly perfect (F1 0.99) while question, greeting and name are lower (0.87, 0.83, 0.71): structured formats are easier for regular expressions than natural language, as the paper claims (absolute values depend on our templates). *Inference:* the proposed approach does not lose accuracy relative to the library; any accuracy gain comes from not applying a lossy input stage.

### *5.4 Summary of Metrics*

TABLE IX
EXISTING VERSUS PROPOSED

| Metric | Existing | Proposed |
|---|---|---|
| Log-rule throughput, 1 thread | 64,778 lines/s | 528,283 lines/s |
| Inputs/s, 5 patterns, batch 10K | 4,691 (fork) | 48,884,218 (DFA) |
| Peak scaling vs. 1 core | 1.72× (fork) | 5.29× (DFA) |
| Mask agreement with regexec | (reference) | 0 mismatches |
| Exact-set accuracy | 0.698 | 0.798 |

## 6. CONCLUSION

This milestone delivers a working regex-to-DFA compiler and a baseline log-classification pipeline, verified against the C library on 10,179 corpus lines and 100,000 random strings, together with a measured reproduction of the base paper. On the same 26 rules the DFA engine is 8.2× faster than the library, grouping keeps the tables at 7,593 states instead of 39,764, and the base paper's process-per-pattern design is 13× slower than a sequential loop. The main limitations seen so far are the start-up cost of DFA construction (617 ms) and false PII labels from digit identifiers when no validation is applied.

The next milestones are: checksum validators and a confidence model for multi-label output; masking of personal data; a multi-core pipeline with line-level parallelism and lock-free queues; correlation of attacks across lines; and evaluation on the full data sets with hand-labelled samples.

## REFERENCES


[1] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, "A survey on automated log analysis for reliability engineering," *ACM Comput. Surv.*, vol. 54, no. 6, Art. no. 130, 2021, doi: 10.1145/3460345.

[2] M. O. Rabin and D. Scott, "Finite automata and their decision problems," *IBM J. Res. Develop.*, vol. 3, no. 2, pp. 114–125, Apr. 1959, doi: 10.1147/rd.32.0114.

[3] R. McNaughton and H. Yamada, "Regular expressions and state graphs for automata," *IRE Trans. Electron. Comput.*, vol. EC-9, no. 1, pp. 39–47, Mar. 1960, doi: 10.1109/TEC.1960.5221603.

[4] K. Thompson, "Programming techniques: Regular expression search algorithm," *Commun. ACM*, vol. 11, no. 6, pp. 419–422, Jun. 1968, doi: 10.1145/363347.363387.

[5] A. V. Aho and M. J. Corasick, "Efficient string matching: An aid to bibliographic search," *Commun. ACM*, vol. 18, no. 6, pp. 333–340, Jun. 1975, doi: 10.1145/360825.360855.

[6] C. Xu, S. Chen, J. Su, S. M. Yiu, and L. C. K. Hui, "A survey on regular expression matching for deep packet inspection: Applications, algorithms, and hardware platforms," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 4, pp. 2991–3029, 2016, doi: 10.1109/COMST.2016.2565660.

[7] S. Kumar, S. Dharmapurikar, F. Yu, P. Crowley, and J. Turner, "Algorithms to accelerate multiple regular expressions matching for deep packet inspection," in *Proc. ACM SIGCOMM*, 2006, pp. 339–350, doi: 10.1145/1159913.1159952.

[8] D. Ficara, A. Di Pietro, S. Giordano, G. Procissi, F. Vitucci, and G. Antichi, "Differential encoding of DFAs for fast regular expression matching," *IEEE/ACM Trans. Netw.*, vol. 19, no. 3, pp. 683–694, Jun. 2011, doi: 10.1109/TNET.2010.2089639.

[9] M. Becchi and P. Crowley, "A-DFA: A time- and space-efficient DFA compression algorithm for fast regular expression evaluation," *ACM Trans. Archit. Code Optim.*, vol. 10, no. 1, Art. no. 4, 2013.

[10] M. Becchi and P. Crowley, "A hybrid finite automaton for practical deep packet inspection," in *Proc. ACM CoNEXT*, 2007, pp. 1–12.

[11] M. Becchi and P. Crowley, "Extending finite automata to efficiently match Perl-compatible regular expressions," in *Proc. ACM CoNEXT*, 2008, doi: 10.1145/1544012.1544037.

[12] A. X. Liu and E. Norige, "A de-compositional approach to regular expression matching for network security," *IEEE/ACM Trans. Netw.*, vol. 27, no. 6, pp. 2179–2191, Dec. 2019, doi: 10.1109/TNET.2019.2941920.

[13] S. Dharmapurikar and J. W. Lockwood, "Fast and scalable pattern matching for network intrusion detection systems," *IEEE J. Sel. Areas Commun.*, vol. 24, no. 10, pp. 1781–1792, Oct. 2006.

[14] X. Wang *et al.*, "Hyperscan: A fast multi-pattern regex matcher for modern CPUs," in *Proc. 16th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2019, pp. 631–648.

[15] J. C. Davis, C. A. Coghlan, F. Servant, and D. Lee, "The impact of regular expression denial of service (ReDoS) in practice: An empirical study at the ecosystem scale," in *Proc. ESEC/FSE*, 2018, pp. 246–256, doi: 10.1145/3236024.3236027.

[16] D. Luchaup, R. Smith, C. Estan, and S. Jha, "Speculative parallel pattern matching," *IEEE Trans. Inf. Forensics Security*, vol. 6, no. 2, pp. 438–451, Jun. 2011, doi: 10.1109/TIFS.2011.2112647.

[17] Y. Ko, M. Jung, Y.-S. Han, and B. Burgstaller, "A speculative parallel DFA membership test for multicore, SIMD and cloud computing environments," *Int. J. Parallel Program.*, vol. 42, pp. 456–489, 2014, doi: 10.1007/s10766-013-0258-5.

[18] T. Mytkowicz, M. Musuvathi, and W. Schulte, "Data-parallel finite-state machines," in *Proc. ASPLOS*, 2014, pp. 529–542, doi: 10.1145/2541940.2541988.

[19] Y.-H. E. Yang and V. K. Prasanna, "Robust and scalable string pattern matching for deep packet inspection on multicore processors," *IEEE Trans. Parallel Distrib. Syst.*, vol. 24, pp. 2283–2292, 2013, doi: 10.1109/TPDS.2012.217.

[20] J. L. Gustafson, "Reevaluating Amdahl's law," *Commun. ACM*, vol. 31, no. 5, pp. 532–533, May 1988, doi: 10.1145/42411.42415.

[21] L. Lamport, "Specifying concurrent program modules," *ACM Trans. Program. Lang. Syst.*, vol. 5, no. 2, pp. 190–222, Apr. 1983.

[22] A. Baumann, J. Appavoo, O. Krieger, and T. Roscoe, "A fork() in the road," in *Proc. Workshop Hot Topics Oper. Syst. (HotOS)*, 2019, doi: 10.1145/3317550.3321435.

[23] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2017, doi: 10.1109/ICWS.2017.13.

[24] P. He, J. Zhu, S. He, J. Li, and M. R. Lyu, "Towards automated log parsing for large-scale log data analysis," *IEEE Trans. Dependable Secure Comput.*, 2018.

[25] M. Landauer *et al.*, "System log clustering approaches for cyber security applications: A survey," *Comput. Secur.*, vol. 92, Art. no. 101739, 2020, doi: 10.1016/j.cose.2020.101739.

[26] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," in *Proc. IEEE Int. Symp. Softw. Rel. Eng. (ISSRE)*, 2023.

[27] M. Landauer, F. Skopik, M. Frank, W. Hotwagner, M. Wurzenberger, and A. Rauber, "Maintainable log datasets for evaluation of intrusion detection systems," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 4, pp. 3466–3482, 2023.

[28] M. Landauer, F. Skopik, M. Wurzenberger, W. Hotwagner, and A. Rauber, "Have it your way: Generating customized log datasets with a model-driven simulation testbed," *IEEE Trans. Rel.*, vol. 70, no. 1, pp. 402–415, Mar. 2021.

[29] M. Dayarathna and S. Perera, "Recent advancements in event processing," *ACM Comput. Surv.*, vol. 51, no. 2, pp. 33:1–33:36, 2018, doi: 10.1145/3170432.

[30] I. Pilán, P. Lison, L. Øvrelid, A. Papadopoulou, D. Sánchez, and M. Batet, "The Text Anonymization Benchmark (TAB): A dedicated corpus and evaluation framework for text anonymization," *Comput. Linguist.*, vol. 48, no. 4, pp. 1053–1101, Dec. 2022, doi: 10.1162/coli_a_00458.

[31] P. Lison, I. Pilán, D. Sánchez, M. Batet, and L. Øvrelid, "Anonymisation models for text data: State of the art, challenges and future directions," in *Proc. 59th Annu. Meeting Assoc. Comput. Linguistics (ACL-IJCNLP)*, 2021, pp. 4188–4203, doi: 10.18653/v1/2021.acl-long.323.

[32] H. P. Gumm, "A new class of check-digit methods for arbitrary number systems," *IEEE Trans. Inf. Theory*, vol. 31, no. 1, pp. 102–105, Jan. 1985, doi: 10.1109/TIT.1985.1056991.

[33] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, 2015, Art. no. 73, doi: 10.1145/2807591.2807644.

[34] A. Roodt, B. K. M. Watling, W. Bester, B. van der Merwe, S. Sung, and Y.-S. Han, "Benchmarking regular expression matching," in *Implementation and Application of Automata (CIAA 2024)* (Lecture Notes in Computer Science, vol. 15015). Cham, Switzerland: Springer, 2024.